重邮802冲刺重点与代码背诵合集

一、冲刺考点与出现频率

频率按2024—2026年802真题回忆资料中的考点组统计；基础补充项单列。

程序阅读与算法设计

考点	出现年份（频率）	复习重点
循环与递归的时间复杂度	2024—2026（3/3）	跟踪循环变量的增减、倍增及内外层依赖，累计基本语句次数；递归分清调用次数、每次工作量和最大深度。算法题另核辅助数组与递归栈空间，不能只数函数有几层。
链表代码补全	2024—2026（3/3）	插入、归并、交集、双链相邻交换、链表排序。先画修改前后的连接，再确定前驱、当前与后继；改链接前保存旧地址，删除后核对前驱是否移动，检查表头、表尾与空表。
栈与队列代码补全	2025、2026（2/3）	循环链式队列入队出队、共享栈。先确认栈顶及队首队尾指向，再推空满条件、读写与移动次序；共享栈抓两端相向增长，循环队列分清牺牲单元和其他判满方式。
二叉树递归阅读	2024、2026（2/3）	统计叶结点、判断平衡并返回高度。逐项核空树返回值、叶结点条件、左右递归及结果合并；识别函数返回的是高度、数量还是状态，注意失衡标记怎样向上传递。
图遍历与拓扑代码	2024、2026（2/3）	2024拓扑排序补全、2026 DFS生成树补全。读懂邻接点枚举和访问标记，区分一棵生成树与整幅图的生成森林；拓扑处理中先统计入度，输出顶点后删除出边并更新零入度集合。
二叉树算法设计	2024—2026（3/3）	判BST、分层统计、最长根叶路径、先序与中序建树。按目标选择中序、后序或层序；分层统计抓层末标记，路径题保留整条路径，建树推准左右区间；BST需约束整棵子树。
图算法设计	2024—2026（3/3）	指定长度路径、经过指定顶点的路径、检验拓扑序列。路径按边数计长，简单路径不能重复顶点，回溯后恢复状态；经指定点须在同次搜索中处理。拓扑验证核范围、重复及当前入度。
链表算法设计	2025、2026（2/3）	倒数第k个结点、约瑟夫环。倒数k维持固定间距并处理表长不足；循环报数维护当前结点及前驱，删除后从下一人重新报1；确认是否带头结点、何时结束及最后结点的释放。

填空与综合应用（手算）

考点	出现年份（频率）	复习重点
散列表与平均查找长度	2024—2026（3/3）	按插入次序构造表，练线性探测、双散列和拉链法；逐项列成功比较次数。开放定址的失败查找探测到空位置为止，拉链法按链上实际比较结点计数；核分母及题设约定，分清装填因子与探测次数。
哈夫曼树与编码	2024—2026（3/3）	每次取当前最小的两项合并，保留完整构造过程；按左右约定写编码，以叶权乘码长之和核WPL，也可用合并权值之和复核。掌握前缀编码及二叉哈夫曼树的结点、叶数关系。
二叉树遍历、重建与表示	2024—2026（3/3）	根据遍历序列划分左右子树、还原树形，再补其他遍历；树森林转换抓左孩子右兄弟。中序线索区分孩子指针与前驱/后继线索，三叉链表看清双亲域；不能只凭先序和后序保证唯一建树。
BST与AVL	2024—2026（3/3）	BST按关键字大小构造、恢复与遍历，按结点层数求成功ASL；AVL逐次插入，找最靠近插入点的失衡祖先并判LL、RR、LR、RL，旋转后重核高度、平衡因子与中序次序。
折半查找	2024—2026（3/3）	按题设统一向下或向上取中点，列查找比较序列并画判定树；成功ASL按关键字比较次数，失败ASL按失败区间和实际比较次数。判定树画出后，另核最坏路径长度。
数组地址计算	2024—2026（3/3）	先确定行优先/列优先、各维长度、下标下界与基准地址，再算元素偏移并乘每元素字节数。行优先按后续维长、列优先按前续维长计算各下标的贡献；区分元素个数、字节偏移及最终地址。
栈、队列状态与序列	2024—2026（3/3）	模拟入栈出栈，判断某输出序列是否可能；求方案数时区分固定输出条件。循环队列先读front/rear定义，推空满与长度公式，再逐次更新指针；所有取模均按题目容量。
表达式的栈处理	2024—2026（3/3）	中缀转后缀按优先级、括号和同级结合方向处理；后缀求值遇运算符先弹右操作数，再弹左操作数。逐步记录栈内状态，减法、除法特别核次序及题目规定的运算规则。
树与森林的结点关系	2024—2026（3/3）	用“各结点度数之和等于边数”和“森林边数=结点数-树数”建立等式；二叉树用叶数与二度结点数关系，非空二叉树的空指针按2n-(n-1)计。完全二叉树再结合层次编号和最后一层规模。
堆调整	2024—2026（3/3）	区分大根堆、小根堆；建堆从最后一个非叶结点向前下滤，插入从新叶向上调整，删除堆顶先用末元素补根。计比较次数时分开“两个孩子比较”和“待调元素与选中孩子比较”。
KMP的next数组	2024、2025（2/3）	按题给下标及next定义，找失配位置之前子串的最长相等真前后缀；nextval按定义继续跳过相同字符。练逐位手算与失配回退位置，分清前缀长度和跳转下标；不背构造或匹配代码。
广义表Head与Tail	2025、2026（2/3）	从最内层运算向外求值，每步保留括号层次；Head取第一个元素，Tail取去掉首元素后的表。区分原子、单元素表、空表，不对原子取Head/Tail；长度只数最外层元素，深度按嵌套层数计。

考点	出现年份（频率）	复习重点
图的最短路径	2024、2026（2/3）	Dijkstra每轮确定当前距离最小的未确定顶点，再松弛并更新前驱；Floyd按中转点顺序写逐轮矩阵，距离和路径信息对应。区分不可达与零权边，掌握适用边权；只练手算，不背代码。
AOE网与关键路径	2025、2026（2/3）	按拓扑顺序求ve、逆拓扑顺序求vl，再算每条活动的e、l与时差；关键活动满足e=l。工期由关键路径决定，可能存在多条；不能把权值最大的边或任意最长单边当关键路径。
BFS、DFS及最小生成树	2025（1/3，综合）	按指定邻接次序写遍历及生成树，非连通时继续选未访问顶点。Prim选树内到树外的最小连接边，Kruskal按边权排序并判环；列所选边和总权值。2026另有阅读题DFS生成树。
排序一趟执行	2024（1/3，综合）	练快排首趟、建堆、希尔/基数手算；先读枢轴、增量或基数约定，每趟单列结果。快排追踪枢轴最终位置，基数各轮分配收集保持相对次序；2025另有阅读题链表排序。
强连通图的最少边数	2026（1/3）	区分有向/无向、连通/强连通，利用每个顶点的入度、出度及度数和推边数界；构造达到下界的图作验证。强连通的两向可达不等于两点必须直接连边，单顶点情况另看。

低频或未直接考到的基础重点

考点	近三年考查情况	复习重点
线性表操作代价与存储选择	2026考链表插入复杂度；综合对比未单列	顺序表按位访问O(1)，插删通常要移动元素；链表按位定位O(n)，已知前驱后的局部插删O(1)。区分“寻找位置”和“修改链接”，理解头结点、头尾指针的作用，按题设判断完整操作代价。
排序性能、稳定性与选型	未见独立选型题	比较插入、冒泡、选择、快排、堆和归并的最好/平均/最坏时间、辅助空间及稳定性；快排计递归栈。会按基本有序、要求稳定、空间受限、链式存储选择方法，能说明所用条件。
图存储、度数与遍历复杂度	存储是图题基础；未见独立比较题	邻接矩阵的行列含义、无向图邻接表的2E个边结点；求入度、出度、边数。整图DFS/BFS用矩阵时O(V²)、邻接表时O(V+E)；图存储分别占O(V²)、O(V+E)，遍历还需O(V)辅助空间。
无权图最短路与连通分量	近三年未见直接设问	BFS以层数给最少边数，首次入队时标记并记录距离/前驱；多源BFS把所有源点同时入队。无向图逐次从未访问顶点发起遍历可数连通分量；普通遍历的访问标记不在回退时清除。
完全二叉树判定与性质区分	未见完整性判定算法题	理解完全、满、平衡和BST各自限制，避免相互替代。按层序判完全性：遇到空孩子位置后，后面不能再有非空结点；特别检查只有右孩子、末层出现空隙等反例，并核空树约定。
链表逆置、拆分重排与快慢指针	这些具体变式未见直接设问	先保存后继再逆置或摘接，拆分后分别处理并接回，保持结点不丢失、不重复。快慢指针找中点要核奇偶表长、起点及前后中点约定；重排时保存两侧后继，注意尾部断链。倒数k另用固定间距。

二、代码背诵与算法模板

背诵顺序：01—08线性结构，09—17树与图的基础及应用，18—19查找，20—23排序。先掌握基本操作，再练组合应用。

全篇约定：代码采用课程常见的C语言风格，保留C++引用参数和bool，整段汇总可按C++17编译；引用参数表示允许修改调用者的变量。不是所有代码块都能脱离公共类型单独编译。考场按题目给定的结构体、下标和函数接口调整。

顺序表、一般数组查找从0起；排序数组从1起且预留第0格。链表通常带头结点，约瑟夫环明确例外；空树高度0，根为第1层。长度非负，指针指向合法对象，各输入/输出数组满足本项容量要求；创建结点时假定内存分配成功。结构定义出现一次，后续直接使用。

复杂度中的n为元素或结点数，h为树高，V、E为顶点数、边数。固定数组的容量是示例上限；比较算法的空间随规模增长时，按容量与所处理规模同阶配置，预留容量与实际栈占用有区别。输入结构、输出结果和额外工作空间分开说明。递归调用栈必须计入，复杂度以这里给出的实现为准。

```
#include <cstdio>
#include <cstdlib>
#include <climits>
const int MaxSize = 1000;
const int MaxVertexNum = 100;
```

01 顺序表按位插入与删除

为什么背：这是读懂数组移动、循环方向和下标边界的基础。先掌握这两段，再读有序插入、区间删除等代码时，只需判断哪些元素要移动。

约定：数据存于 data[0..length-1]，题目中的第 i 个元素位序从1起。MaxSize 是数组容量，初始化只把表长清零，不必清空整个数组。

```
typedef struct {
    int data[MaxSize];
    int length;
} SqList;

void InitSqList(SqList &L) {
    L.length = 0;                    // 只有表长以内的数据有效
}

bool SqInsert(SqList &L, int i, int x) {
    if (i < 1 || i > L.length + 1) return false;
    if (L.length == MaxSize) return false;
```

```
代码续
    // 从后往前搬，避免覆盖尚未移动的元素
    for (int j = L.length; j >= i; --j)
        L.data[j] = L.data[j - 1];
    L.data[i - 1] = x;                // 位序i对应下标i-1
    ++L.length;
    return true;
}

bool SqDelete(SqList &L, int i, int &x) {
    if (i < 1 || i > L.length) return false;
    x = L.data[i - 1];                // 先保存被删元素
    // 从前往后补位，末尾失效数据不用清零
    for (int j = i; j < L.length; ++j)
        L.data[j - 1] = L.data[j];
    --L.length;
    return true;
}
```

复杂度与易错点：插入、删除最坏时间均为 O(n)，辅助空间 O(1)。尾插、尾删为 O(1)。插入合法位序有 n+1 个，删除只有 n 个；不要把位序直接用作数组下标。

02 链表建表销毁与双链相邻交换

为什么背：头插、尾插是后面逆置、归并、拆表的基本动作。建表时就应弄清输入次序与结果次序的关系，而不是每遇到一种链表题重新背整段算法。

约定：普通单链表带头结点，尾指针为 NULL。输入数组为 a[0..n-1]，n>=0；建表前 L 不拥有待保留的旧链。为突出算法，以下建表假定 malloc 成功。销毁时连头结点一起释放。

```
typedef struct LNode {
    int data;
    struct LNode *next;
} LNode, *LinkList;

void CreateByHead(LinkList &L, const int a[], int n) {
    L = (LNode *)malloc(sizeof(LNode));
    L->next = NULL;                   // 先建立空表
    for (int i = 0; i < n; ++i) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = a[i];
        s->next = L->next;            // 先接住原首结点
        L->next = s;                  // 再把新结点挂到表头
    }
}
```

```
代码续
void CreateByTail(LinkList &L, const int a[], int n) {
    L = (LNode *)malloc(sizeof(LNode));
    L->next = NULL;
    LNode *rear = L;                  // 空表的尾就是头结点
    for (int i = 0; i < n; ++i) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = a[i];
        s->next = NULL;
        rear->next = s;               // 接到当前表尾
        rear = s;                     // 新结点成为表尾
    }
}

void DestroyList(LinkList &L) {
    while (L != NULL) {
        LNode *next = L->next;        // 释放前保存后继
        free(L);
        L = next;
    }                                 // 返回时L已经是NULL
}
```

复杂度与易错点：两种建表均为 O(n) 时间，结果链占 O(n) 空间，除结果链外只需 O(1) 辅助空间。销毁为 O(n) 时间、O(1) 辅助空间。头插得到输入的逆序，尾插保持输入顺序；不能先释放结点再读它的 next。

双链表补充：交换p与其前一个数据结点。2025程序阅读出现过相邻交换。这里使用带头双链表，p必须有数据前驱；p可以是尾结点。先保存原前驱和后继，再改六处连接，避免覆盖旧地址后形成自环。

```
typedef struct DNode {
    int data;
    struct DNode *prior, *next;
} DNode;

bool SwapWithPrev(DNode *p) {
    if (p == NULL || p->prior == NULL ||
        p->prior->prior == NULL) return false;
    DNode *q = p->prior;              // q是原来的数据前驱
    DNode *before = q->prior;
    DNode *after = p->next;           // 尾结点的后继可以为空
    before->next = p;
    p->prior = before;
    p->next = q;
    q->prior = p;
    q->next = after;
    if (after != NULL) after->prior = q;
    return true;
}
```

交换操作时间、辅助空间均为 O(1)。上面判定依赖头结点的 prior==NULL；不能直接用于循环双链表或把头结点当作待交换的数据结点。

03 普通链表与循环链表原地逆置

为什么背：这是“保存后继后改链接”的典型模板，可用于链表重组和逆序输出前的结构调整。普通版与循环版对照着记，能避免将循环链表写成永不结束的遍历。

约定：两版都带头结点。普通空表满足 L->next==NULL；循环空表满足 L->next==L。不交换数据、不新建数据结点。

```
void ReverseList(LinkList L) {
    if (L == NULL) return;
    LNode *p = L->next;                // 保存待处理链的入口
    L->next = NULL;                    // 结果链先置空
    while (p != NULL) {
        LNode *next = p->next;         // 改next前先保存后继
        p->next = L->next;             // 当前结点头插到结果链
        L->next = p;
        p = next;                     // 转向原链的下一个结点
    }
}

void ReverseCircularList(LinkList L) {
    if (L == NULL) return;
    LNode *p = L->next;
    L->next = L;                       // 空的结果链仍然是环
    while (p != L) {                  // 回到头结点才算遍历完
        LNode *next = p->next;
        p->next = L->next;
        L->next = p;
        p = next;
    }
}
```

复杂度与易错点：两版均为 O(n) 时间、O(1) 辅助空间。循环版先令头结点指回自己，第一次头插时旧首结点便接到头结点，最后它自然成为尾结点；不能把循环版的结束条件写成 p!=NULL。这里使用原链的旧 next 推进，不沿已经反转的指针继续走。

04 用前驱指针删除所有满足条件的结点

为什么背：“扫描、判断、删除、继续”能迁移到按值删除、去掉特定区间元素、保留某类结点等任务。关键不在判断式，而在删除后前驱是否移动。

约定：带头结点的普通单链表。示例删除所有 data>x 的数据结点，保留其他结点原顺序，头结点保留。

```
void DeleteGreater(LinkList L, int x) {
    if (L == NULL) return;
    LNode *pre = L;                  // pre始终是p的前驱
    LNode *p = L->next;
    while (p != NULL) {
        if (p->data > x) {
            pre->next = p->next;      // 先接通剩余链
            free(p);                 // 再释放被删结点
            p = pre->next;           // 下一结点仍需判断
        } else {
            pre = p;                 // 保留当前结点才移动pre
            p = p->next;
        }
    }
}
```

复杂度与易错点：O(n) 时间、O(1) 辅助空间。连续多个结点都应删除时，pre 必须留在原处；若每轮都后移前驱，就会漏删或访问已释放的地址。换删除条件时，先确认题目是否要求释放结点、是否允许修改原链。

05 复用结点归并两个有序单链表

为什么背：有序链表归并训练比较两个当前结点、摘取结点、处理剩余链。它也是有序交集、去重和分解题的起点。递增归并与递减归并的差别，要落实到尾插和头插的连接方式。

约定：A、B 是两个互不共享结点的带头普通单链表，数据均非递减排列。重复值保留；结果存回A，释放B的头结点并将B置空。以下两版是二选一，不能先用一版合并后再把失效的B传入另一版。

```
bool MergeAscending(LinkList &A, LinkList &B) {
    if (A == NULL || B == NULL || A == B) return false;
    LNode *pa = A->next, *pb = B->next;
    LNode *rear = A;                 // 复用A的头结点
    while (pa != NULL && pb != NULL) {
        if (pa->data <= pb->data) {
            rear->next = pa;         // 较小者接到结果表尾
            pa = pa->next;
        } else {
            rear->next = pb;
            pb = pb->next;
        }
        rear = rear->next;
    }
```

```
代码续
    // 剩余链已然有序，可直接接上
    rear->next = (pa != NULL) ? pa : pb;
    free(B);                          // 只释放多余的头结点
    B = NULL;
    return true;
}
```

若结果要求非递增，依然每次取较小者，但改为头插。剩余链也要逐个头插，不能直接接在结果尾部。

```
bool MergeDescending(LinkList &A, LinkList &B) {
    if (A == NULL || B == NULL || A == B) return false;
    LNode *pa = A->next, *pb = B->next;
    A->next = NULL;                   // A成为空的结果链
    while (pa != NULL || pb != NULL) {
        LNode *s;
        // pb已空，或pa存在且不大于pb，就取pa
        if (pb == NULL ||
            (pa != NULL && pa->data <= pb->data)) {
            s = pa;
            pa = pa->next;            // 先推进，再改s的next
        } else {
            s = pb;
            pb = pb->next;
        }
        s->next = A->next;            // 头插反转取出次序
        A->next = s;
    }
    free(B);
    B = NULL;
    return true;
}
```

复杂度与易错点：若两表长度为m、n，两版最坏时间均为 O(m+n)，辅助空间 O(1)。只要要求复用原结点，就不能重新申请一整条数据链。递减版会反转相等元素的取出次序，不能因此宣称它稳定。

06 固定间距找倒数第k个与倍速找中点

为什么背：2025算法题出现过倒数第k个结点；找中点则是链表分段、重排的常用步骤。两者都用两个指针，但前者维持固定距离，后者维持速度差，不能混写成一种走法。

约定：带头结点普通单链表。倒数第k从1起，非法k或表长不足返回 NULL；中点函数在偶数长度时返回前一个中点，空表返回 NULL。

```
LNode *KthFromEnd(LinkList L, int k) {
    if (L == NULL || k <= 0) return NULL;
    LNode *slow = L->next;
    LNode *fast = L->next;            // 两者从首个数据结点起
    for (int i = 0; i < k; ++i) {
        if (fast == NULL) return NULL;
        fast = fast->next;           // fast先走k步
    }
    // 此处fast为NULL也合法：恰好k等于表长
    while (fast != NULL) {
        slow = slow->next;           // 此后两者都每次走一步
        fast = fast->next;
    }
    return slow;
}

LNode *FrontMiddle(LinkList L) {
    if (L == NULL || L->next == NULL) return NULL;
    LNode *slow = L->next, *fast = L->next;
    // fast至少还能走两条边时才继续
    while (fast->next != NULL &&
           fast->next->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

复杂度与易错点：两者均为 O(n) 时间、O(1) 辅助空间。用长度为1、2、5、6的小表检查中点停点；用 k=1、k=n、k=n+1 检查倒数查找。指针从头结点或首元结点出发都可以设计正确算法，但推进步数和失败判断必须与起点一致。

07 循环链表模拟约瑟夫报数

为什么背：2026算法题出现过约瑟夫环。背这一版是为了掌握“在环上维护当前结点和前驱、删除后重新起数”，而不是只记住一个幸存者公式。

约定：n个人编号1至n，从第s个人报1，报到m者出列，下一个人重新报1，返回最后幸存者编号。这里的环**不带头结点**；输入需满足 n>=1、1<=s<=n、m>=1，否则返回0。建环假定内存分配成功。

```
int Josephus(int n, int s, int m) {
    if (n < 1 || s < 1 || s > n || m < 1) return 0;
    LNode *first = (LNode *)malloc(sizeof(LNode));
    first->data = 1;
    LNode *rear = first;
    for (int i = 2; i <= n; ++i) {
        LNode *q = (LNode *)malloc(sizeof(LNode));
        q->data = i;
        rear->next = q;
        rear = q;
    }
    rear->next = first;                // 尾接首，完成建环

    LNode *p = first, *pre = rear;     // pre始终在p前面
    for (int i = 1; i < s; ++i) {
        pre = p;
        p = p->next;                   // 定位到第s个人
    }
    while (p->next != p) {             // 自环表示只剩一人
        for (int count = 1; count < m; ++count) {
            pre = p;
            p = p->next;               // 当前p已经报了1
        }
        pre->next = p->next;           // 越过报到m的结点
        free(p);
        p = pre->next;                 // 下一人重新从1报
    }
    int answer = p->data;
    free(p);                           // 最后一个结点也要释放
    return answer;
}
```

复杂度与易错点：建环 O(n)，报数删除最坏 O(nm)，整体 O(nm)；本函数自己创建环，辅助空间为 O(n)。若题目已经给出环，单看删除阶段只需 O(1) 辅助空间。m=1 时每次立即删除当前人；循环终止不能检查 NULL。题目若要求出列顺序，在 free(p) 前记录或输出该编号。

08 顺序栈与循环队列

为什么背：栈队状态在近三年填空中反复出现，队列也进入过程序阅读。树的非递归遍历、层序遍历及BFS要把它们当工具使用，先背清指针定义，才不会将不同版本的操作拼在一起。

约定：下面元素类型为 int；需要存树结点时应把元素类型改为树结点指针。顺序栈 top=-1 为空，top指向栈顶元素；栈容量为 MaxSize。

```
typedef struct {
    int data[MaxSize];
    int top;
} SqStack;

void InitStack(SqStack &S) {
    S.top = -1;
}

bool StackEmpty(const SqStack &S) {
    return S.top == -1;
}

bool Push(SqStack &S, int x) {
    if (S.top == MaxSize - 1) return false;
    S.data[++S.top] = x;              // 先移动，再写入
    return true;
}

bool Pop(SqStack &S, int &x) {
    if (S.top == -1) return false;
    x = S.data[S.top--];              // 先取值，再移动
    return true;
}

bool GetTop(const SqStack &S, int &x) {
    if (S.top == -1) return false;
    x = S.data[S.top];                // 读取但不出栈
    return true;
}
```

循环队列中front指向队头元素，rear指向下一次入队的位置；牺牲一个单元区分空满，因此有效容量是MaxSize-1。

```
typedef struct {
    int data[MaxSize];
    int front, rear;
} SqQueue;
```

```
代码续
void InitQueue(SqQueue &Q) {
    Q.front = Q.rear = 0;
}

bool QueueEmpty(const SqQueue &Q) {
    return Q.front == Q.rear;
}

int QueueLength(const SqQueue &Q) {
    return (Q.rear - Q.front + MaxSize) % MaxSize;
}

bool EnQueue(SqQueue &Q, int x) {
    if ((Q.rear + 1) % MaxSize == Q.front) return false;
    Q.data[Q.rear] = x;               // rear就是待写位置
    Q.rear = (Q.rear + 1) % MaxSize;
    return true;
}

bool DeQueue(SqQueue &Q, int &x) {
    if (QueueEmpty(Q)) return false;
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;
    return true;
}

bool GetFront(const SqQueue &Q, int &x) {
    if (QueueEmpty(Q)) return false;
    x = Q.data[Q.front];              // 读取不改变队头
    return true;
}
```

复杂度与易错点：各基本操作均为 O(1) 时间、O(1) 辅助空间；容器自身按容量C占 O(C) 存储。循环队列长度要加容量再取模；front==rear 是空，不能同时当成满。若题目改了front/rear或top的含义，先重推边界和操作次序，不能照搬本版。

共享双栈补充：2026程序阅读出现过共享双栈。栈1从数组左端向右增长，栈2从右端向左增长；两个栈顶相邻才是整个共享空间已满。

```
typedef struct {
    int data[MaxSize];
    int top1, top2;
} SharedStack;

void InitSharedStack(SharedStack &S) {
    S.top1 = -1;
    S.top2 = MaxSize;
}
```

```
代码续
bool SharedPush(SharedStack &S, int which, int x) {
    if (which != 1 && which != 2) return false;
    if (S.top1 + 1 == S.top2) return false;
    if (which == 1) S.data[++S.top1] = x;
    else S.data[--S.top2] = x;
    return true;
}

bool SharedPop(SharedStack &S, int which, int &x) {
    if (which == 1) {
        if (S.top1 == -1) return false;
        x = S.data[S.top1--];
    } else if (which == 2) {
        if (S.top2 == MaxSize) return false;
        x = S.data[S.top2++];
    } else return false;
    return true;
}
```

共享双栈每次操作也是 O(1) 时间、O(1) 辅助空间。总元素数为 top1+1+MaxSize-top2；判空需分别检查两栈，不是检查两个栈顶是否相等。

09 树递归与返回状态

为什么背：树统计、树高和递归功能判断都从这套写法变化。2024阅读涉及数叶，2026阅读涉及平衡二叉树求高；先背清空树返回值、子树递归和结果合并，比逐题记函数更有效。

约定：空树高度为0，根所在层为1；树无环，每个结点只属于一个父结点。下列类型供后续树算法共用。

```
typedef struct BiTNode {
    int data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;

int TreeHeight(BiTree T) {
    if (T == NULL) return 0;           // 空树没有层
    int left = TreeHeight(T->lchild);
    int right = TreeHeight(T->rchild);
    return (left > right ? left : right) + 1;
}

int TreeNodeCount(BiTree T) {
    if (T == NULL) return 0;
    return TreeNodeCount(T->lchild)
         + TreeNodeCount(T->rchild) + 1; // 加当前结点
}
```

```
代码续
int TreeLeafCount(BiTree T) {
    if (T == NULL) return 0;
    if (T->lchild == NULL && T->rchild == NULL)
        return 1;                    // 当前结点就是叶子
    return TreeLeafCount(T->lchild)
         + TreeLeafCount(T->rchild);  // 内部结点不额外加1
}

// 返回非负高度；返回-1表示该子树不平衡。
int BalancedHeight(BiTree T) {
    if (T == NULL) return 0;
    int left = BalancedHeight(T->lchild);
    if (left == -1) return -1;        // 不把失败值当高度
    int right = BalancedHeight(T->rchild);
    if (right == -1) return -1;
    if (left - right > 1 || right - left > 1)
        return -1;
    return (left > right ? left : right) + 1;
}
```

复杂度与易错处：各函数最坏时间O(n)，递归栈O(h)。判平衡时一次后序递归同时得到高度和状态，不在每个结点重新调用求高度函数。树退化成链时h=n，不能一律写O(log n)空间。

回看：数据结构强化第4节P3—P4、P8—P9；《代码总结》树与二叉树的基本操作。

10 非递归中序遍历

为什么背：它把“递归暂存返回位置”具体变成入栈、出栈，适合程序补全，也为非递归树题提供起点。与普通递归遍历相比，重点记住出栈以后转向哪棵子树。

约定：结点数不超过MaxSize，out数组容量不少于结点数，返回写入的元素个数。数组栈存放结点指针。

```
int InOrderIter(BiTree T, int out[]) {
    BiTree stack[MaxSize];
    int top = -1, count = 0;
    BiTree p = T;
    while (p != NULL || top != -1) {  // 两者都空才结束
        while (p != NULL) {
            stack[++top] = p;        // 根暂存，先处理左边
            p = p->lchild;
        }
        p = stack[top--];            // 左子树已经处理完
        out[count++] = p->data;      // 中序：左、根、右
        p = p->rchild;               // 进入右子树后再向左走
    }
    return count;
}
```

复杂度与易错处：时间O(n)，栈最多同时保存O(h)个指针。代码用固定容量数组预留栈，实际分配O(MaxSize)；按n设置容量时为O(n)。不要把外层条件写成逻辑与，也不要在出栈后再次转向左孩子。

回看：数据结构基础第7节；《代码总结》树遍历。

11 层序遍历与分层求宽度

为什么背：2024算法题的分层统计与本模板同源。宽度、层数、某层结点数和按层加权，都需要正确识别“一层何时结束”。

约定：结点数不超过MaxSize；宽度指同一层实际存在的结点数，不把空位置计入。

```
int TreeWidth(BiTree T) {
    if (T == NULL) return 0;
    BiTree queue[MaxSize];
    int front = 0, rear = -1;
    queue[++rear] = T;
    int last = rear;                // 当前层最后结点下标
    int width = 0, best = 0;
    while (front <= rear) {
        BiTree p = queue[front++];  // 每次出队处理一个结点
        ++width;
        if (p->lchild != NULL)
            queue[++rear] = p->lchild;
        if (p->rchild != NULL)
            queue[++rear] = p->rchild;
        if (front > last) {         // 原来这一层全部出队
            if (width > best) best = width;
            width = 0;              // 下一层重新计数
            last = rear;            // 此时队尾就是下一层末尾
        }
    }
    return best;
}
```

复杂度与易错处：时间O(n)。这是不回收前端位置的线性数组队列，按n配置容量时辅助空间O(n)，不能直接写成O(w)。若题目按层号加权，在层结束处使用width计算，并在同一位置更新层号；具体统计式以题目定义为准。

回看：数据结构强化第5节P2—P5。

12 中序携带前值判断BST

为什么背：2024算法题直接涉及BST判断。这里的“跨递归层共享前一个访问值”还可以用于有序性检查和中序相邻元素处理。

约定：采用关键字互不相同的严格BST定义；若题目允许相等，必须先明确相等键的放置规则，不能只把比较符号机械改掉。

```
bool CheckBSTInOrder(BiTree T, int &prev, bool &hasPrev) {
    if (T == NULL) return true;
    if (!CheckBSTInOrder(T->lchild, prev, hasPrev))
        return false;
    if (hasPrev && T->data <= prev)
        return false;               // 中序必须严格递增
    prev = T->data;                  // 给后续访问保存前值
    hasPrev = true;
    return CheckBSTInOrder(T->rchild, prev, hasPrev);
}

bool JudgeBST(BiTree T) {
    int prev = 0;                   // 尚未使用，不是数值哨兵
    bool hasPrev = false;
    return CheckBSTInOrder(T, prev, hasPrev);
}
```

复杂度与易错处：最坏时间O(n)，递归栈O(h)。prev和hasPrev必须在递归各层共享。用hasPrev区分“尚无前驱”，因此结点值等于INT_MIN时仍正确；只比较根与两个直接孩子不能保证整棵树是BST。

回看：数据结构强化第5节P9—P10。

13 先序与中序递归建树

为什么背：2026算法题出现先中序建树。核心是用中序中的根位置划分左右规模，再推导先序区间；中序加后序也按同样的区间思想调整。

约定：pre与in是同一棵树的合法先序、中序序列，关键字互异；数组下标从0开始，结点分配成功。调用入口为BuildPreIn(pre,0,in,0,n)。

```
BiTree BuildPreIn(const int pre[], int preL,
                  const int in[], int inL, int n) {
    if (n == 0) return NULL;
    int rootValue = pre[preL];       // 先序区间首元素是根
    int k = inL;
    while (in[k] != rootValue) ++k;  // 合法且互异保证能找到
    int leftSize = k - inL;
    int rightSize = n - leftSize - 1;
    BiTree root = (BiTree)malloc(sizeof(BiTNode));
    root->data = rootValue;
```

```
代码续
    root->lchild = BuildPreIn(
        pre, preL + 1, in, inL, leftSize);
    root->rchild = BuildPreIn(
        pre, preL + leftSize + 1, in, k + 1, rightSize);
    return root;
}
```

复杂度与易错处：每次在线性区间中找根，最坏时间O(n²)，递归栈O(h)，新建结果树占O(n)。不能把这种实现写成O(n)。右子树先序起点为preL+leftSize+1；有重复关键字时，仅凭两种序列不一定能唯一建树，本代码不处理该情形。

回看：数据结构基础第7节、强化第5节P11。

14 根到叶的最长路径

为什么背：2025算法题涉及最长根叶路径。它能训练递归中的路径数组、层号以及“找到答案后向上传递成功状态”。

约定：路径长度按结点数计；path由调用者提供，容量至少为树高。多条最长路径取先找到的一条。本实现先求高度，再找达到该高度的根叶路径，避免每次发现更长路径就整段复制。

```
bool FindPathAtDepth(BiTree T, int depth,
                     int target, int path[]) {
    if (T == NULL) return false;
    path[depth - 1] = T->data;       // 当前层的路径位置
    if (T->lchild == NULL && T->rchild == NULL)
        return depth == target;     // 必须在叶结点结束
    if (FindPathAtDepth(T->lchild, depth + 1, target, path))
        return true;                // 保留找到的整条路径
    return FindPathAtDepth(T->rchild, depth + 1, target, path);
}

int LongestRootLeafPath(BiTree T, int path[]) {
    int height = TreeHeight(T);      // 第09项的递归求高
    if (height == 0) return 0;
    FindPathAtDepth(T, 1, height, path);
    return height;                  // 有效结果为path[0..height-1]
}
```

复杂度与易错处：两遍遍历的总时间仍为O(n)，辅助递归栈O(h)，输出路径占O(h)。未成功分支留下的数组值会被后续分支覆盖，只有成功返回后的前height个元素属于答案。若采用每次更新最优值就复制路径的另一写法，需另计复制成本，不能不加分析地写O(n)。

回看：数据结构强化第5节P6—P8；本项保留路径模型，采用两遍遍历控制复制成本。

15 邻接表DFS与BFS

为什么背：图算法题与程序阅读都需要会遍历。2025综合应用考遍历，2026阅读涉及DFS生成树；DFS递归与BFS入队标记是许多图题的基础。

约定：顶点下标为0至vexnum-1，0≤vexnum≤MaxVertexNum；邻接表结点的adjvex均合法。输出order容量至少为V，存顶点下标而非顶点的数据值；整个图遍历要处理不连通情况。

```
typedef struct ArcNode {
    int adjvex;
    struct ArcNode *nextarc;
} ArcNode;

typedef struct VNode {
    int data;
    ArcNode *firstarc;
} VNode;

typedef struct {
    VNode vertices[MaxVertexNum];
    int vexnum, arcnum;
} ALGraph;

void DFSCollect(const ALGraph &G, int u, bool visited[],
                int order[], int &count) {
    visited[u] = true;              // 先标记，防止沿环再次进入
    order[count++] = u;
    for (ArcNode *p = G.vertices[u].firstarc;
         p != NULL; p = p->nextarc) {
        int v = p->adjvex;
        if (!visited[v]) DFSCollect(G, v, visited, order, count);
    }
}

int DFSForest(const ALGraph &G, int order[]) {
    bool visited[MaxVertexNum] = {false};
    int count = 0;
    for (int u = 0; u < G.vexnum; ++u)
        if (!visited[u]) DFSCollect(G, u, visited, order, count);
    return count;
}

int BFSForest(const ALGraph &G, int order[]) {
    bool visited[MaxVertexNum] = {false};
    int queue[MaxVertexNum], front = 0, rear = 0;
    int count = 0;
    for (int s = 0; s < G.vexnum; ++s) {
        if (visited[s]) continue;
        visited[s] = true;
        queue[rear++] = s;
```

```
代码续
        while (front < rear) {
            int u = queue[front++];
            order[count++] = u;
            for (ArcNode *p = G.vertices[u].firstarc;
                 p != NULL; p = p->nextarc) {
                int v = p->adjvex;
                if (!visited[v]) {
                    visited[v] = true; // 入队时标记，不等出队
                    queue[rear++] = v;
                }
            }
        }
    }
    return count;
}
```

复杂度与易错处：邻接表时间O(V+E)，按顶点规模配置visited、队列时辅助空间O(V)，DFS递归栈也最多O(V)。普通遍历的visited不在回退时清除；如果改成邻接矩阵，逐行扫描的时间为$O(V^2)$。遍历序列受邻接点访问顺序影响。

回看：数据结构基础第9节、强化第6节；《代码总结》图的遍历。

16 回溯搜索指定长度的简单路径

为什么背：2024算法题的指定长度路径，需要区分“图遍历一次”和“尝试不同路径”。visited表示当前路径已经使用的顶点，失败后要恢复。

约定：使用第15项的邻接表；k按边数计。求s到t的一条恰含k条边的简单路径，顶点不重复；path容量不少于V，成功时path[0..k]有效。s=t且k=0允许单顶点路径。

```
bool SearchExactPath(const ALGraph &G, int u, int t,
                     int depth, int k, bool used[], int path[]) {
    used[u] = true;
    path[depth] = u;
    if (depth == k) {
        bool found = (u == t);
        used[u] = false;             // 每一层返回都恢复状态
        return found;
    }
    if (u != t) {                    // 不能先到终点再绕回来
        for (ArcNode *p = G.vertices[u].firstarc;
             p != NULL; p = p->nextarc) {
            int v = p->adjvex;
```

```
代码续
            if (!used[v] && SearchExactPath(
                    G, v, t, depth + 1, k, used, path)) {
                used[u] = false;
                return true;
            }
        }
    }
    used[u] = false;                // 让其他候选路径可再用u
    return false;
}

bool FindExactPath(const ALGraph &G, int s, int t,
                   int k, int path[]) {
    if (s < 0 || s >= G.vexnum || t < 0 || t >= G.vexnum)
        return false;
    if (k < 0 || k >= G.vexnum) return false;
    bool used[MaxVertexNum] = {false};
    return SearchExactPath(G, s, t, 0, k, used, path);
}
```

复杂度与易错处：辅助空间O(V)。若搜索实际产生S次递归调用，每次最多扫描d条邻接记录，时间为O(V+S(d+1))；最坏可能枚举大量候选简单路径，不能套O(V+E)。d是最大出度，V项来自状态初始化。若改为“经过指定顶点”，应在同一次搜索中携带是否经过它的状态；两段各自简单的路径拼接后可能重复顶点。

回看：数据结构强化第6节路径部分；复杂度按本项实际回溯过程分析。

17 入度法拓扑排序与序列验证

为什么背：2024阅读涉及拓扑补码，2026算法题要求验证给定拓扑序列。两者共用“处理一个顶点后，删除它的出边”这一动作，区别在于谁决定下一个顶点。

约定：G为有向图。拓扑排序输出order，其容量至少为V；返回false时order只含已输出的部分序列。验证时seq恰有V项，必须逐项检查范围、重复和当前入度。不能用“没有遇到负入度”替代合法性判断。

```
void ComputeIndegree(const ALGraph &G, int indegree[]) {
    for (int u = 0; u < G.vexnum; ++u) indegree[u] = 0;
    for (int u = 0; u < G.vexnum; ++u)
        for (ArcNode *p = G.vertices[u].firstarc;
             p != NULL; p = p->nextarc)
            ++indegree[p->adjvex];
}
```

```
代码续
bool TopologicalOrder(const ALGraph &G, int order[]) {
    int indegree[MaxVertexNum];
    ComputeIndegree(G, indegree);
    int queue[MaxVertexNum], front = 0, rear = 0;
    for (int u = 0; u < G.vexnum; ++u)
        if (indegree[u] == 0) queue[rear++] = u;
    int count = 0;
    while (front < rear) {
        int u = queue[front++];
        order[count++] = u;
        for (ArcNode *p = G.vertices[u].firstarc;
             p != NULL; p = p->nextarc) {
            int v = p->adjvex;
            if (--indegree[v] == 0) queue[rear++] = v;
        }
    }
    return count == G.vexnum;        // 未全部输出说明有环
}

bool VerifyTopoOrder(const ALGraph &G, const int seq[]) {
    int indegree[MaxVertexNum];
    bool used[MaxVertexNum] = {false};
    ComputeIndegree(G, indegree);
    for (int i = 0; i < G.vexnum; ++i) {
        int u = seq[i];
        if (u < 0 || u >= G.vexnum || used[u]) return false;
        if (indegree[u] != 0) return false; // 前驱尚未处理完
        used[u] = true;
        for (ArcNode *p = G.vertices[u].firstarc;
             p != NULL; p = p->nextarc)
            --indegree[p->adjvex];
    }
    return true;
}
```

复杂度与易错处：均为O(V+E)时间、按顶点规模配置时O(V)辅助空间。验证题不必另外生成一条拓扑序列再比较：合法序列可能不唯一。判断拓扑序唯一性时，要检查每轮所有当前可选零入度顶点是否恰好一个，并确认最终处理了V个顶点。

回看：数据结构强化第6节拓扑部分；《代码总结》入度和拓扑排序。

18 折半查找

为什么背：近三年都出现折半查找过程、判定树或ASL相关任务。循环边界和区间收缩也经常用于代码阅读；必须先知道什么条件允许丢弃一半区间。

约定：A[0..n-1]非递减，返回任意一个相等元素的下标，失败返回-1；本代码不保证返回重复元素的第一个位置。

```
int BinarySearch(const int A[], int n, int key) {
    int low = 0, high = n - 1;     // 闭区间，两端都可能是答案
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (A[mid] == key) return mid;
        if (A[mid] < key)
            low = mid + 1;         // mid及其左侧都不再考虑
        else
            high = mid - 1;        // mid及其右侧都不再考虑
    }
    return -1;                     // low>high，候选区间已空
}
```

复杂度：O(log n)时间、O(1)辅助空间；空表直接失败。**记忆点：**闭区间配low≤high、mid±1。二分每轮利用有序条件排除区间，不能把快排的双指针划分也当作同一种二分过程。

19 二叉排序树的查找插入与删除

为什么背：BST是近三年树形查找的主干。查找与插入把“向哪个孩子走”落实成代码；删除的三种情况能帮助完成指针填空和构造过程。优先默写查找、插入，删除重点记双孩子时如何接续。

约定：复用第09项的BiTree；关键字互异，左小右大。插入重复值返回false，删除不存在的值返回false。结点由malloc创建，与free配对。

```
BiTree BSTSearch(BiTree T, int key) {
    while (T != NULL && T->data != key) {
        if (key < T->data) T = T->lchild;
        else T = T->rchild;
    }
    return T;                      // NULL表示查找失败
}

bool BSTInsert(BiTree &T, int key) {
    if (T == NULL) {
        T = (BiTree)malloc(sizeof(BiTNode));
        T->data = key;
```

```
代码续
        T->lchild = T->rchild = NULL;
        return true;
    }
    if (key == T->data) return false;
    if (key < T->data) return BSTInsert(T->lchild, key);
    return BSTInsert(T->rchild, key);
}

bool BSTDelete(BiTree &T, int key) {
    if (T == NULL) return false;
    if (key < T->data) return BSTDelete(T->lchild, key);
    if (key > T->data) return BSTDelete(T->rchild, key);

    BiTree old = T;
    if (T->lchild == NULL) {       // 叶子也属于这一种情况
        T = T->rchild;            // 用右孩子或NULL接替
        free(old);
    } else if (T->rchild == NULL) {
        T = T->lchild;            // 仅有左孩子
        free(old);
    } else {
        BiTree parent = T;
        BiTree s = T->rchild;     // 找右子树的最小结点
        while (s->lchild != NULL) {
            parent = s;
            s = s->lchild;
        }
        T->data = s->data;        // 后继值覆盖被删关键字
        if (parent == T)
            parent->rchild = s->rchild;
        else
            parent->lchild = s->rchild;
        free(s);                  // 后继没有左孩子，直接接右孩子
    }
    return true;
}
```

复杂度：树高h，三种操作均为O(h)时间；迭代查找辅助空间O(1)，这里递归插入、删除为O(h)。普通BST最坏可退化为单链，不能直接写O(log n)。**记忆点：**删除两个孩子的结点时只是替换关键字，真正释放的是后继结点；若结点还有关联记录，应连同记录一起搬移。

20 直接插入排序与冒泡排序

为什么背：这两种代码短，循环界限和元素移动非常适合程序阅读，也用于区分比较、移动与提前结束。先掌握一趟做什么，再推总复杂度。

约定：排序代码统一使用A[1..n]，A[0]预留为暂存位；数组至少n+1格。下面冒泡每趟从左向右，把最大值送到当前末尾；题目若规定相反扫描方向，不能套同一个中间序列。

```
void InsertSort(int A[], int n) {
    for (int i = 2; i <= n; ++i) {
        A[0] = A[i];              // 暂存待插入元素，也是哨兵
        int j = i - 1;
        while (A[0] < A[j]) {     // 到j=0时比较必然停止
            A[j + 1] = A[j];      // 大于待插值的元素后移
            --j;
        }
        A[j + 1] = A[0];          // 填入腾出的空位
    }
}

void BubbleSort(int A[], int n) {
    for (int end = n; end > 1; --end) {
        bool swapped = false;
        for (int j = 1; j < end; ++j) {
            if (A[j] > A[j + 1]) {
                int t = A[j];
                A[j] = A[j + 1];
                A[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break;      // 整趟无交换，说明已经有序
    }
}
```

复杂度：二者最好O(n)、平均/最坏O(n²)，辅助空间O(1)，上述实现均稳定。**记忆点：**相等元素不交换或不后移才保留原相对次序；“序列有序时O(n)”对冒泡依赖swapped提前结束，不适用于所有冒泡写法。

21 快速排序的划分与递归

为什么背：Partition是短而完整的指针扫描模型；手算一趟、补空、判断递归范围都要理解它。比起背多种枢轴选法，先把教材的首元素枢轴、挖坑填数法写准确。

约定：A[1..n]，首元素作枢轴；Partition只在low<high时调用。比较中的等号保证有重复值时指针仍能推进。

```
int Partition(int A[], int low, int high) {
    A[0] = A[low];                // 枢轴暂存，low处形成空位
    while (low < high) {
        while (low < high && A[high] >= A[0]) --high;
        A[low] = A[high];         // 从右找到小值，填左边的坑
        while (low < high && A[low] <= A[0]) ++low;
        A[high] = A[low];         // 从左找到大值，填右边的坑
    }
    A[low] = A[0];                // 两指针相遇，枢轴最终落位
    return low;
}

void QuickSort(int A[], int low, int high) {
    if (low >= high) return;      // 空段或单元素段不再递归
    int p = Partition(A, low, high);
    QuickSort(A, low, p - 1);     // 枢轴已就位，不再包含p
    QuickSort(A, p + 1, high);
}
```

首次调用QuickSort(A,1,n)。单趟划分O(n)时间、O(1)辅助空间；整体平均O(nlog n)，最坏O(n^2)。递归栈平均O(log n)、最坏O(n)，不稳定。**记忆点：**首元素枢轴遇升序、降序或全相等输入都可能退化；不能把平均栈深当最坏空间。划分后两侧各自不保证有序。

22 大根堆调整与堆排序

为什么背：堆插入、比较次数、建堆和排序过程是常见手算内容；记住HeapAdjust能统一解释“选哪个孩子、何时停止、空位放在哪里”。

约定：A[1..n]；结点k的孩子为2k、2k+1。下沉前，k的左右子树已经分别是大根堆；调整不负责修复任意位置的错误。

```
void HeapAdjust(int A[], int k, int n) {
    A[0] = A[k];                  // 保存待下沉的值
    for (int child = 2 * k; child <= n; child = 2 * k) {
        if (child < n && A[child + 1] > A[child])
            ++child;              // 右孩子存在且更大，选右孩子
        if (A[0] >= A[child]) break;
        A[k] = A[child];          // 较大孩子上移到父位置
        k = child;                // 空位下移，继续检查下一层
    }
    A[k] = A[0];                  // 暂存值填入最终空位
}
```

代码续

```
void HeapSort(int A[], int n) {
    for (int k = n / 2; k >= 1; --k)
        HeapAdjust(A, k, n);      // 从最后一个非叶结点逆序建堆
    for (int end = n; end > 1; --end) {
        int t = A[1];
        A[1] = A[end];
        A[end] = t;               // 最大值进入有序区
        HeapAdjust(A, 1, end - 1);// 调整时排除已经就位的末尾
    }
}
```

复杂度：单次下沉O(log n)，自底向上建堆O(n)，堆排序O(nlog n)，辅助空间O(1)，不稳定。**记忆点：**大根堆排序得到升序；小根堆得到降序。堆只约束父子关系，不要求兄弟之间有序。

23 二路归并与逆序对计数

为什么背：归并的“三指针、两段收尾、复制回原数组”是经典教材代码；增加一个累计量即可处理逆序对，是已讲过且值得保留的迁移模型。2020逆序对是笔记中的补充题源，不计入近三年出现次数。

约定：A[1..n]；B是同样大小的辅助数组。逆序对为i<j且A[i]>A[j]，相等不算。函数会把A改成升序，并返回原序列的逆序对数；只要归并排序时，忽略返回值即可。

```
long long MergeAndCount(int A[], int B[],
                        int low, int mid, int high) {
    int i = low, j = mid + 1, k = low;
    long long count = 0;
    while (i <= mid && j <= high) {
        if (A[i] <= A[j]) {
            B[k++] = A[i++];      // 相等取左侧，保持稳定
        } else {
            B[k++] = A[j++];
            count += mid - i + 1;// 左侧剩余元素全大于刚取的值
        }
    }
    while (i <= mid) B[k++] = A[i++];
    while (j <= high) B[k++] = A[j++];
    for (int p = low; p <= high; ++p)
        A[p] = B[p];              // 只复制当前合并区间
    return count;
}
```

```
代码续
long long MergeSortCount(int A[], int B[], int low, int high) {
    if (low >= high) return 0;
    int mid = low + (high - low) / 2;
    long long left = MergeSortCount(A, B, low, mid);
    long long right = MergeSortCount(A, B, mid + 1, high);
    long long cross = MergeAndCount(A, B, low, mid, high);
    return left + right + cross;  // 左半、右半、跨两段
}

long long CountInversions(int A[], int n) {
    if (n < 2) return 0;
    int *B = (int *)malloc((n + 1) * sizeof(int));
    long long count = MergeSortCount(A, B, 1, n);
    free(B);                      // 整次排序只申请一份辅助数组
    return count;
}

void MergeSort(int A[], int n) {
    CountInversions(A, n);        // 普通归并排序忽略计数结果
}
```

复杂度：排序和计数均O(nlog n)时间；B占O(n)、递归栈O(log n)，合计O(n)辅助空间，排序稳定。**记忆点：**计数用long long；核心是mid-i+1，不是只加1。若只考普通归并，可去掉计数变量及累加语句，保留同一合并过程。

资料依据：考点频率据2024—2026年802真题回忆资料，题源位置见近三年802任务证据。代码参考知识笔记、代码总结、当前课程与严蔚敏教材，统一接口并补充注释。